

조운호

Platform Engineer

반복 작업을 자동화하는 데서 시작해, 시스템의 책임과 실패 경계를 설계했습니다.

홈랩 운영의 반복을 줄이기 위해 자동화를 시작했습니다. 범위가 커지며 생긴 결함을 직접 겪은 뒤, VM 생성·애플리케이션 배포·사용자 데이터의 수명주기를 분리했습니다.

`code.penguin.yoon@gmail.com`

`github.com/CodingPenguin-yoon`

`yunman.page`

01 반복 운영

02 통합 자동화

03 책임 결함 발견

04 경계 분리

05 실패 시 기존 상태 보존

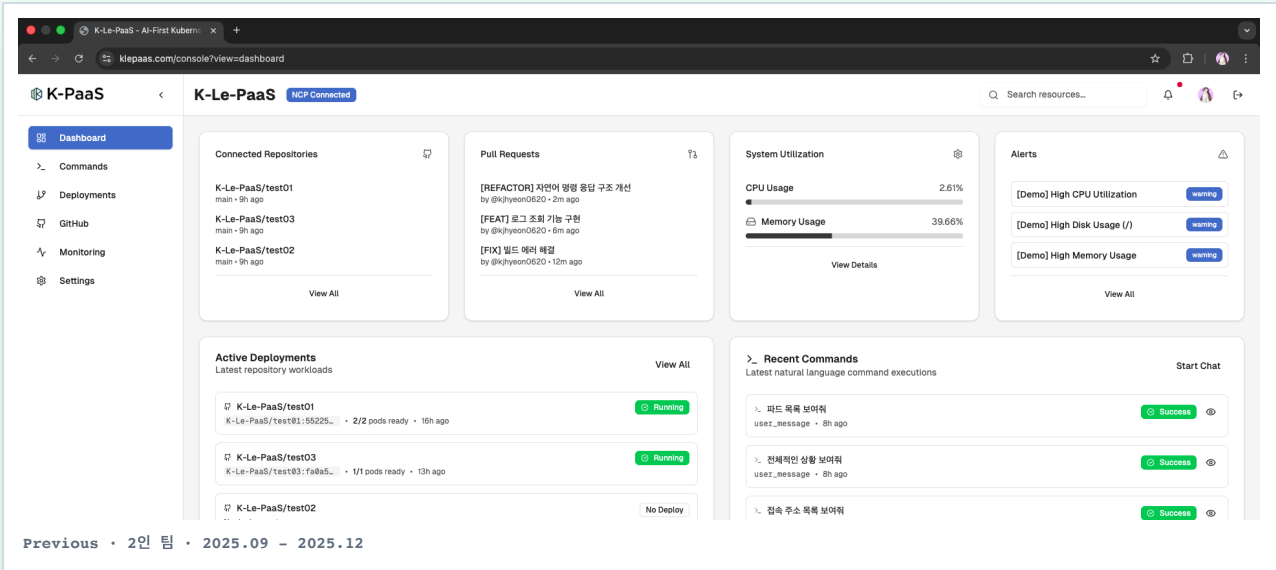
새 서비스를 올릴 때마다, 같은 실행 환경을 다시 만들었습니다.

홈랩에 프로그램을 올리는 일은 애플리케이션 배포만으로 끝나지 않았습니다. VM과 네트워크를 준비하고 Docker와 내부 실행 환경을 구성한 뒤에야 배포를 시작할 수 있었습니다.

처음의 목표는 거대한 플랫폼이 아니라,
매번 반복하던 준비 작업을 줄이는 것이었습니다.

01	02	03
VM 생성	IP·네트워크	Docker
매번 다른 사양과 설정	충돌과 연결 상태 확인	반복 설치와 기본 구성
04	05	6
내부 환경	배포	새 프로젝트
프로젝트마다 다른 실행 조건	실패 지점 재추적	처음부터 다시 시작

자연어 요청을 바로 실행하지 않고, 실행 가능한 계획으로 바꿨습니다.



자연어 요청의 의도와 대상을 해석하고, 실행 가능한 **CommandPlan**으로 정규화한 뒤 Kubernetes·NCP 작업에 연결했습니다.

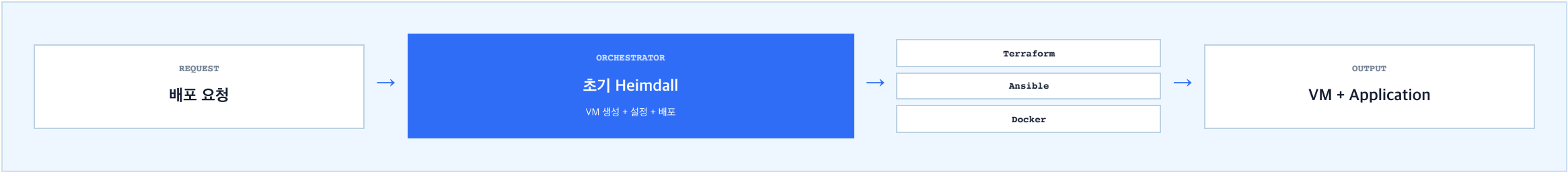


확인된 개인 기여

- Gemini 의도·엔티티 해석
- Ingress 도메인·배포 URL 동기화
- Kubernetes 상태 조회·재시작 CommandPlan
- Prometheus 기반 NKS 모니터링

입력과 실행 사이에 계획과 확인 단계를 두는 관점이 올랩 자동화의 출발점이 됐습니다.

처음에는 모든 책임을 한곳에 모아야 더 효율적이라고 생각했습니다.



01

한 번의 요청

수동 단계를 하나의 실행 경로로 연결

02

반복 감소

VM과 환경 구성을 자동화

03

한 경로에서 추적

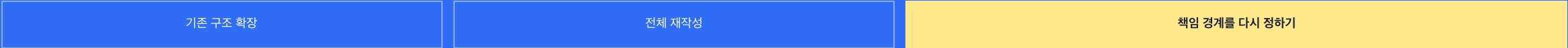
상태와 로그를 한곳에 기록

당시에는 합리적인 선택이었지만, 자동화 범위와 함께 시스템이 책임져야 할 실패도 커졌습니다.

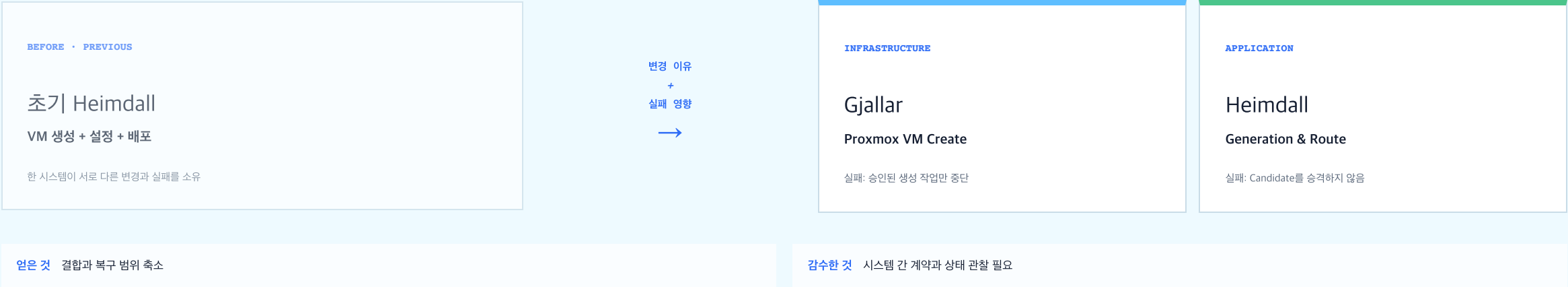
기능을 더할수록 책임의 소유자와 실패의 원인이 흐려졌습니다.

VM 생성과 애플리케이션 배포는 변경되는 이유가 달랐습니다. 인프라 용량과 네트워크를 바꾸는 일, 코드와 설정을 배포하는 일이 같은 실행 경로에서 실패했습니다.

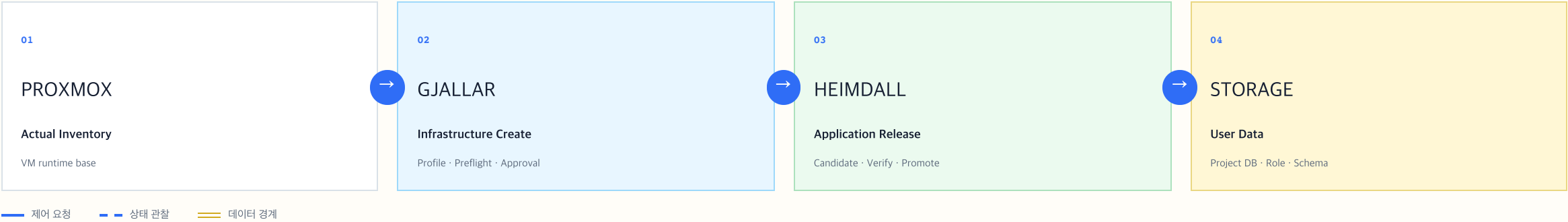
Terraform·Ansible·Docker 의존성을 함께 관리하면서 기능을 추가할수록 복구 범위도 함께 넓어졌습니다.



변경 이유와 실패 영향을 기준으로 Gjallar와 Heimdall을 분리했습니다.



제어·배포·데이터의 수명주기를 같은 실패 정책으로 다루지 않습니다.



핵심 불변 조건 한 시스템의 실패가 다른 시스템의 복구 경로와 사용자 데이터까지 오염시키지 않게 합니다.

실행 전에 실제 상태와 계획을 확인하고, 승인된 변경만 Proxmox에 전달합니다.

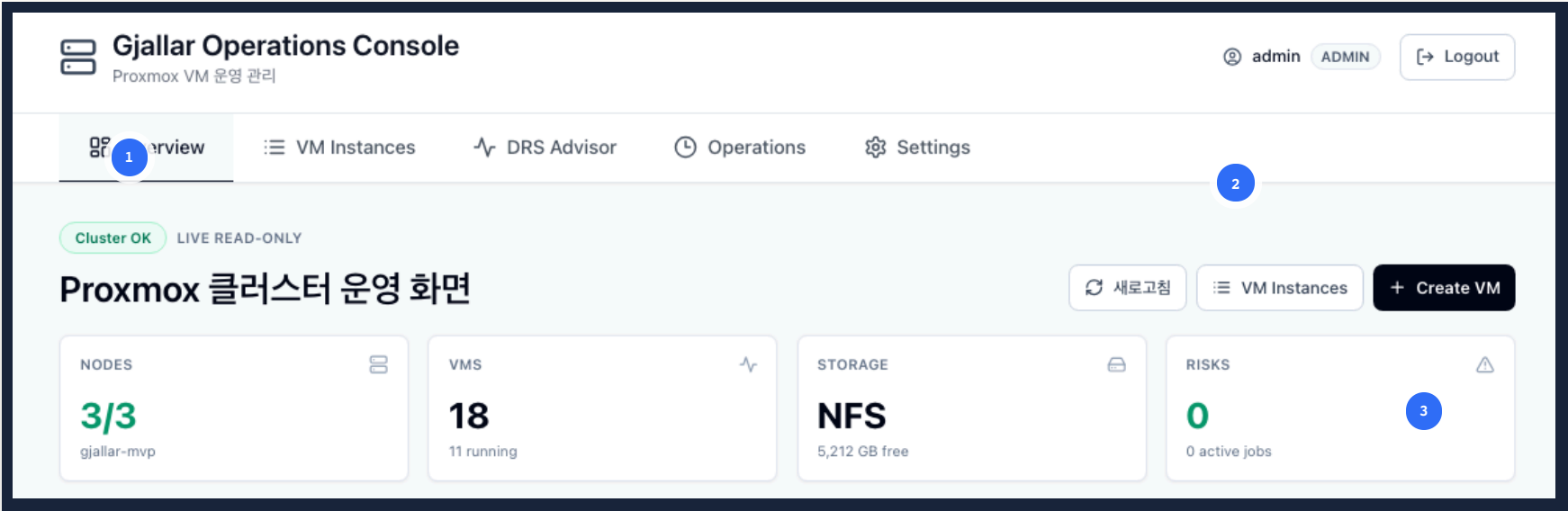
현재 문제에는 Terraform·Ansible을 active dependency로 유지하는 것보다 Proxmox API 기반 Native Create가 더 직접적이었습니다.

도구를 포기한 것이 아니라, 실제 상태와 승인 경계를 직접 제어하기 위해 다시 선택했습니다.

CURRENT SCOPE · Native Create + limited gated Start



실행 결과가 아니라, 실행 전후의 상태를 운영 근거로 남겼습니다.



- 1 Actual Inventory
37개 노드와 18개 VM의 실제 상태
- 2 Storage & Network
생성 전 용량과 연결 조건 확인
- 3 Job Evidence
요청·승인·UPID·observed-after 기록

새 배포를 바로 교체하지 않고, 검증된 Candidate만 Current로 승격합니다.



실패했을 때 무엇을 지킬지부터 각 단계의 복구 경계를 정했습니다.

BUILD / HEALTH

Candidate cleanup

Current 유지

ACTIVATION

Last-known-good config 복원

이전 route 유지

WORKER

DB · marker · label reconcile

불확실한 candidate 보존



불확실한 상태는 자동 삭제보다 보존을 선택합니다.

NOT IMPLEMENTED DB backup / restore · data rollback · credential rotation · automatic purge

Commit·배포 세대·Health·Route를 하나의 화면에서 확인합니다.

- 1

Exact Commit

현재 배포의 입력을 고정
- 2

Generation State

candidate와 current를 구분
- 3

Data Boundary

이미지 rollback과 DB 복구를 분리

HEIMDALL

Preview Deployment Console

PREVIEW SERVICES API base: http://127.0.0.1:18180

Projects

Deployments

Settings

PROJECTS1

NO LIVE PREVIEW0

AUTO DEPLOY1

DEPLOYING0

FAILING0

Projects

Table-first project list. Select a row to inspect and operate on it.

Register project

PROJECT	PROVIDER	BRANCH	STATUS	PREVIEW TARGET	LAST DEPLOY	AUTO	ACTIONS
gjallr gjallr	github	main	SUCCESS	http://127.0.0.1:18002 real preview active	2026. 6. 17. 오후 11:38:05	On	Deploy preview Toggle auto

Project Detail

gjallr

REAL PREVIEW ACTIVE

Releases / Rollback

Image rollback changes preview containers only; managed PostgreSQL data is not restored.

반복을 줄이고, 경계를 나누며, 실패해도 되돌아갈 상태를 남깁니다.

- 01

Observe before change

실행 전에 실제 상태와 계획을 확인합니다.
- 02

Separate by failure

변경 이유가 다르면 책임과 실패 경계를 분리합니다.
- 03

Preserve uncertainty

불확실한 상태는 자동 삭제보다 보존합니다.

PLATFORM / INFRA

Proxmox · Kubernetes · Docker · Linux

BACKEND / DATA

FastAPI · PostgreSQL · Redis · REST API

AUTOMATION

GitHub Actions · Terraform · Ansible · WireGuard

EDUCATION

광운대학교 전자통신공학과 · 2026.02

CERTIFICATIONS

정보처리기사 · 리눅스마스터 2급

조운호 · Platform Engineer

code.penguin.yoon@gmail.com

github.com/CodingPenguin-yoon